Indian Vehicle Classification

Under supervision of:

Prof. Luisa Verdoliva

Made by:

Ibrahim Abdellatif D1800069

Sameh Alshaarawy D18000042

Zeyad Gharaf D18000059

Course: IVPAD 2025

Presentation Agenda

Aim of Project

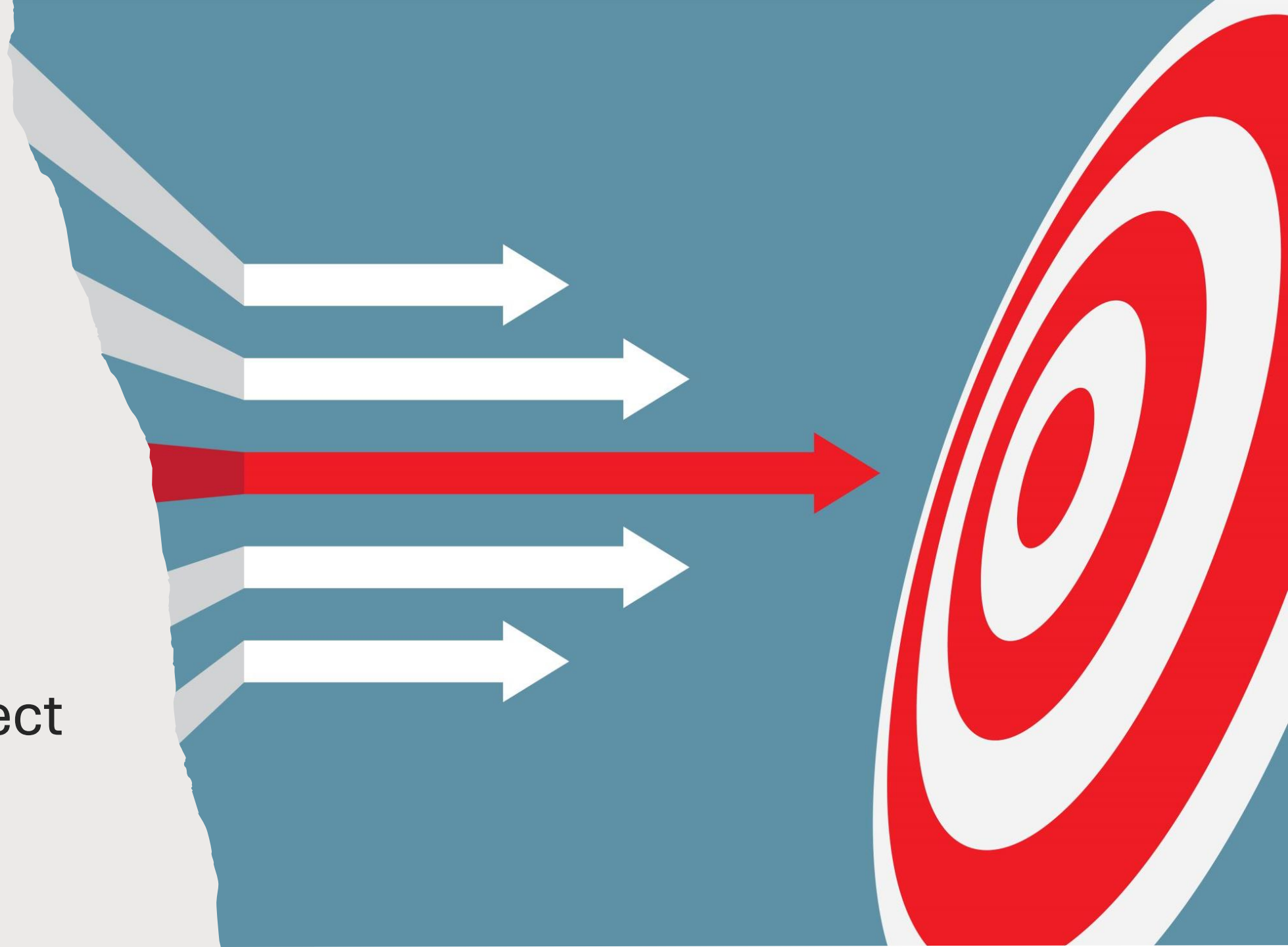
Dataset Overview

Methodology

Performance Analysis

Real-case Study

Aim of project



Aim of the project

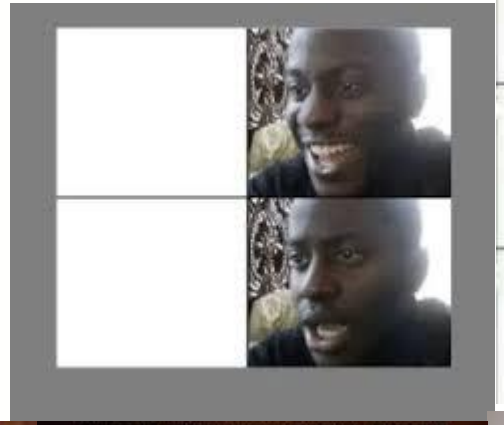
- Classification of the vehicles to 7 classes:

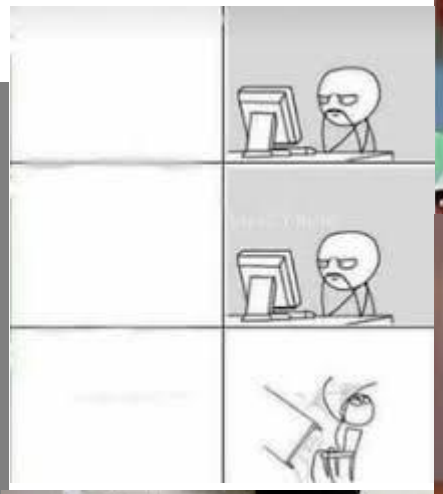
Classes:

Auto
Bike
Car
Motorcycle
Plane
Ship
Train

Scenario

Do you think that Images is important?



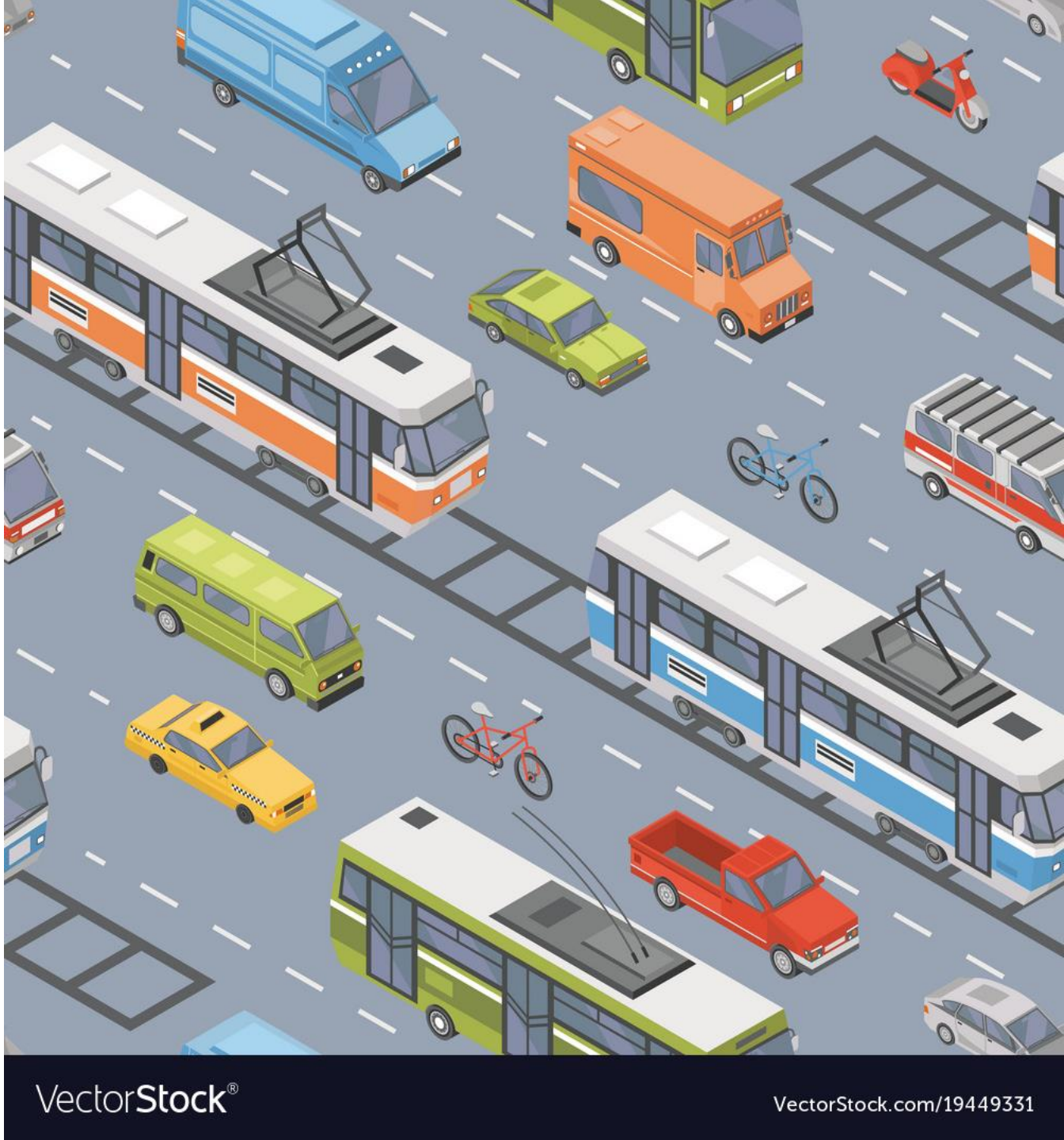


Classification Scenario

Imagine a smart city deploying an **automated traffic monitoring system** using roadside cameras. These cameras are designed to **classify vehicles**—such as cars, motorcycles, buses, trucks, and emergency vehicles—in real time.

Usage:

- Traffic Flow Optimization
- Detecting emergency vehicles
- Lane Enforcement



Motivation

- Automated vehicle classification from images is essential for smart traffic systems, autonomous driving, and urban planning. Traditional methods struggle with accuracy in real-world conditions, while deep learning models can learn complex visual features for reliable classification.
- This project aims to develop an image classification model to accurately identify vehicle types, enhancing traffic monitoring and intelligent transportation solutions.

Dataset Overview

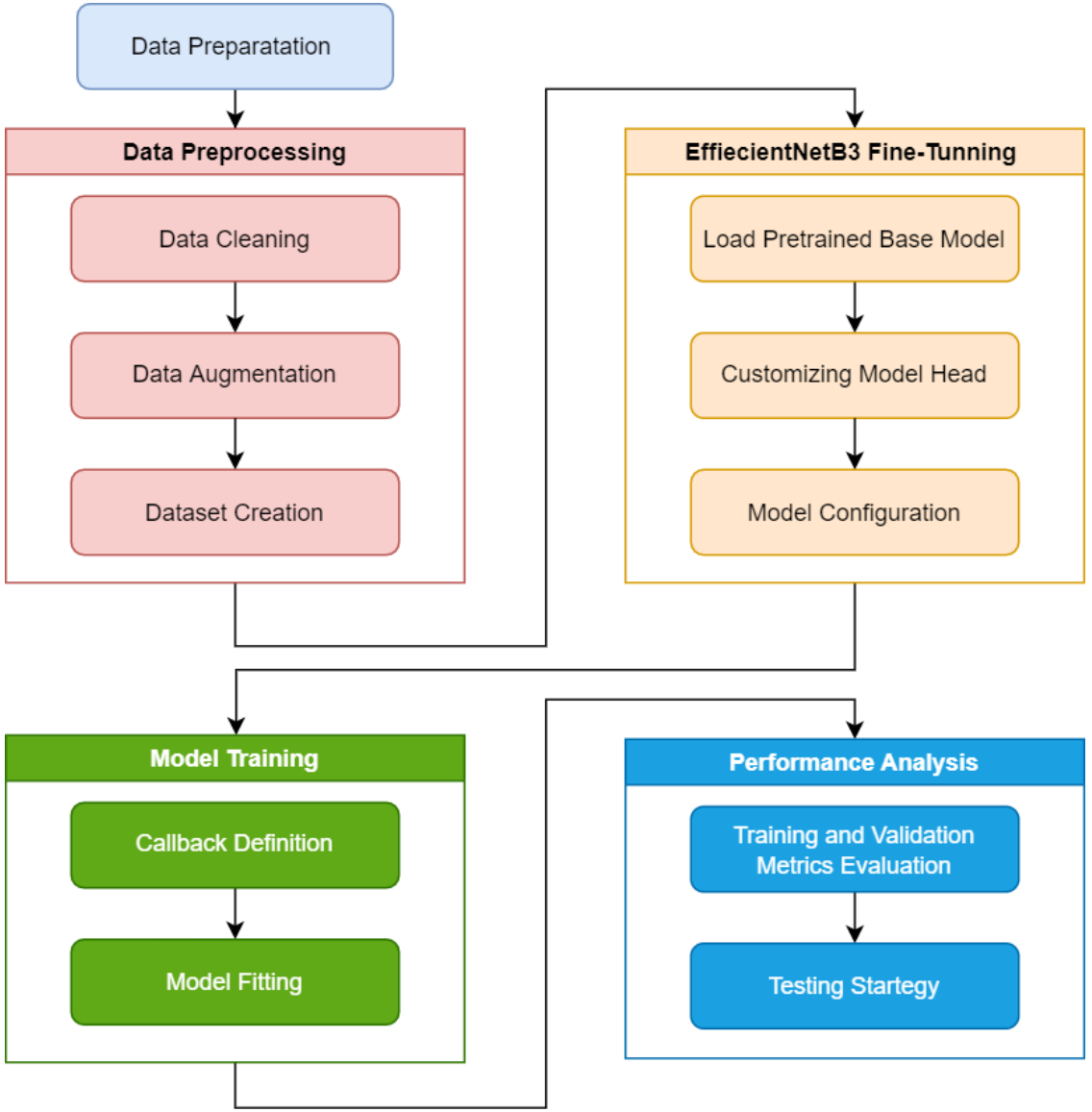
Dataset Overview

- **Overview:** This dataset is designed for vehicle classification tasks and contains a total of 5,600 images distributed across seven categories. Each category represents a different type of vehicle.
- **Image Format:** All images are in JPEG, PNG format with the .jpg and .png extension.
- **folders:**
  - Auto Rickshaws (800 images)
  - Bikes (800 images)
  - Cars (800 images)
  - Motorcycles (800 images)
  - Planes (800 images)
  - Ships (800 images)
  - Trains (800 images)

Source: https://www.kaggle.com/datasets/mohamedmaher5/vehicle-classification

Methodology

Methodology Overview



I. Data Preparation

Objective:
Organize raw image data into a structured format suitable for training, validation, and testing.

Steps Involved:

Data Extraction:
Unzipped the original dataset containing raw image files.

Directory Structuring:
Randomly split the dataset into three subsets:

70% Training
15% Validation
15% Testing

Dataframe Generation:
Created structured DataFrames linking each image to its corresponding label and subset.

II. Data Preprocessing –Data Cleaning

- **Objective:**
  Ensure all images are valid and compatible with the training pipeline by removing corrupted or unsupported files.
- **Cleaning Strategy:**
  - Iterated through all image paths in training, validation, and testing sets.
  - Attempted to open each image and load it fully using PIL.
  - Validated format and mode:
    - Allowed: RGB mode
    - Allowed formats: JPEG, PNG, BMP, GIF
    - Removed: CMYK, WEBP, unreadable, or corrupted files
- **Outcome:**
  - Cleaned dataframes by filtering out flagged entries:
    - train_df_cleaned
    - val_df_cleaned
    - test_df_cleaned

Data Preprocessing – Data Augmentation & Dataset Creation

Data Augmentation

The goal is to enrich training data with made-up variety to improve model generalization.

- **Applied Techniques:**
  - Random Rotation (±20°)
  - Random Zoom (70%–130%)
  - Random Horizontal Flip

Dataset Creation

The goal is to prepare optimized, normalized image datasets for training and validation.

- **Dataset Configuration:**
  - Input dimensions: 320 × 320
  - Batch size: 16 images
  - Normalization: Enabled
- **Labels:**

One-Hot Encoded (categorical mode)

- **Pipeline:**
  - Training set augmented (train_augment)
  - Validation set processed without augmentation

EfficientNetB3 Fine-Tuning

A. Load Base Model

- Pretrained EfficientNetB3 loaded with ImageNet weights
- Top layers excluded → allows custom classification head
- Input shape: (320, 320, 3) for RGB images
- No global pooling applied initially

B. Customizing Model Head

- GlobalAveragePooling2D layer added
- Final Dense layer with 7 units (softmax for multi-class output)
- Architecture wrapped in a Sequential model

C. Model Configuration

- Freeze Pretrained Layers:
  - First 200 layers of base model frozen :retain learned features
  - Remaining layers are trainable: enables task-specific tuning
- Compile Model
  - Loss Function: Categorical Crossentropy
  - Optimizer: Nadam (LR = 1e-4)
  - Evaluation Metric: Accuracy

Model Training

A. Define Callbacks

To improve model performance and prevent overfitting:

- Early Stopping
  - Monitors validation loss
  - Stops training if no improvement for 10 epochs
  - Restores best weights
- Reduce LR On Plateau
  - Reduces learning rate when validation loss plateaus
  - Decrease factor: 0.5
  - Minimum LR: 1e-6
- Model Check point
  - Saves the model with best validation accuracy

B. Train and Validate Model

- Model trained for up to 50 epochs
- Inputting Cleaned & augmented training dataset, Validation dataset
- All callbacks applied to optimize learning

Performance Analysis

Hyperparameter Optimization

Train	Val	test	freezed layer	batch	learning rate
61%	75%	72.69%	350	16	1*10^-4
84.31%	91.37%	87.40%	250	16	1.10^-4
95.09%	93.65%	90.68%	200	16	1*10^-4
93.20%	93.60%	89.50%	200	8	1*10^-4

Training and Validation Metrics Evaluation

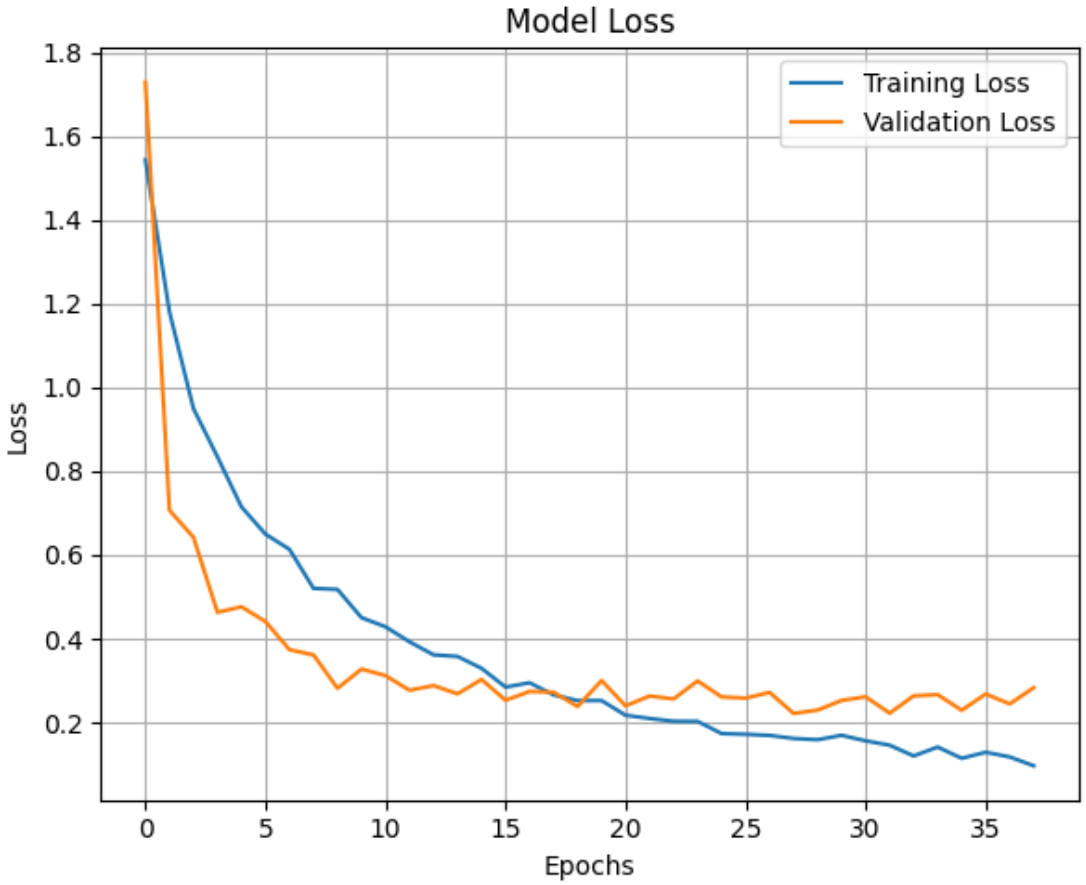
Last epoch

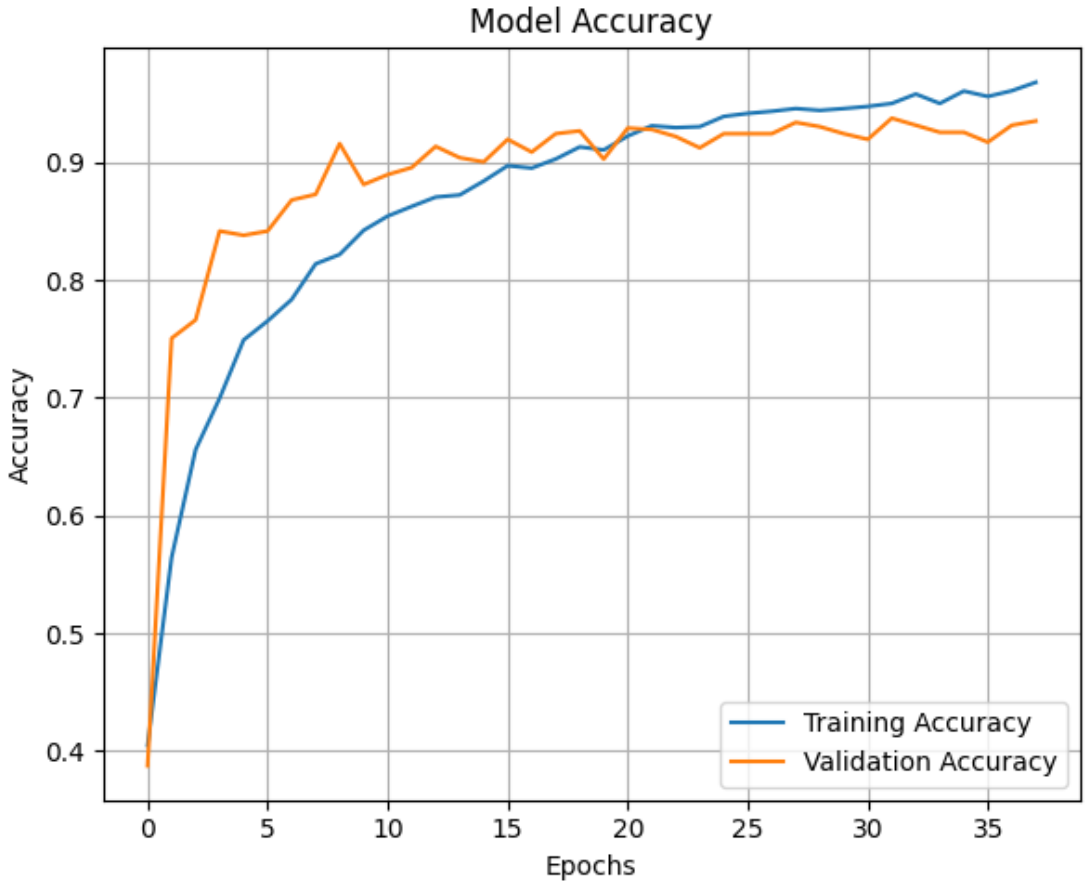
```
Epoch 38: val_accuracy did not improve from 0.93765
244/244 ━━━━━━━━━━━━━━━━━━━━ 111s 453ms/step - accuracy: 0.9688 - loss: 0.0942 - val_accuracy: 0.9353 - val_loss: 0.2830 - learning_rate: 1.0000e-04
Epoch 38: early stopping
Restoring model weights from the end of the best epoch: 28.
<keras.src.callbacks.history.History at 0x79f9fefb7bd0>
```

Best epoch

```
Epoch 28: val_accuracy improved from 0.92926 to 0.93405, saving model to best_model.h5
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save_model(model)`. This file format is considered legacy.
244/244 ━━━━━━━━━━━━━━━━━━━━ 111s 450ms/step - accuracy: 0.9472 - loss: 0.1628 - val_accuracy: 0.9341 - val_loss: 0.2216 - learning_rate: 1.0000e-04
Epoch 29/50
```

Training and Validation Metrics Evaluation





Testing strategy

```
52/52 ━━━━━━━━━━━━━━━━━━━━ 14s 263ms/step - accuracy: 0.9183 - loss: 0.2652
Test loss: 0.30683064460754395
Test accuracy: 0.9105199575424194
```

Internally from dataset:

We have used an image from each class in the test dataset that include:

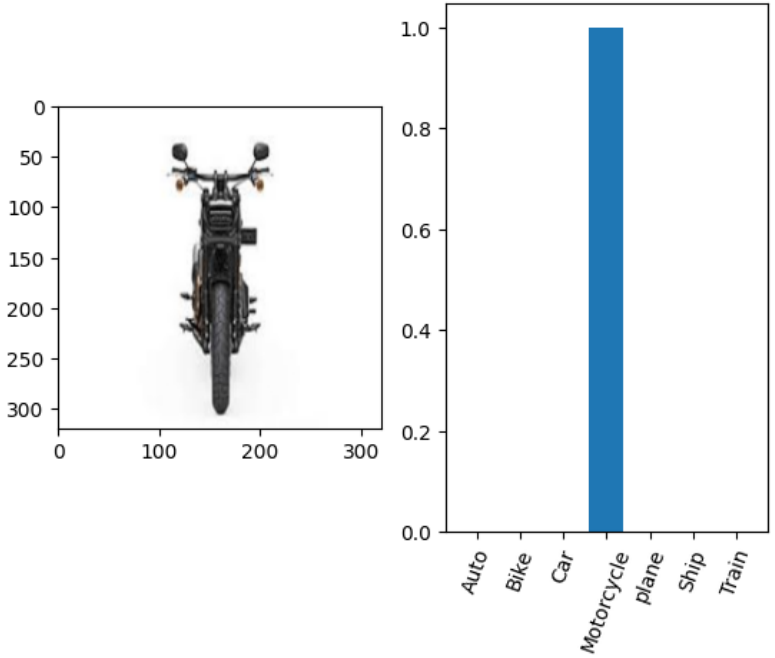
- Car
- Auto
- Bike
- Motorcycle
- Plane
- Ship
- Train

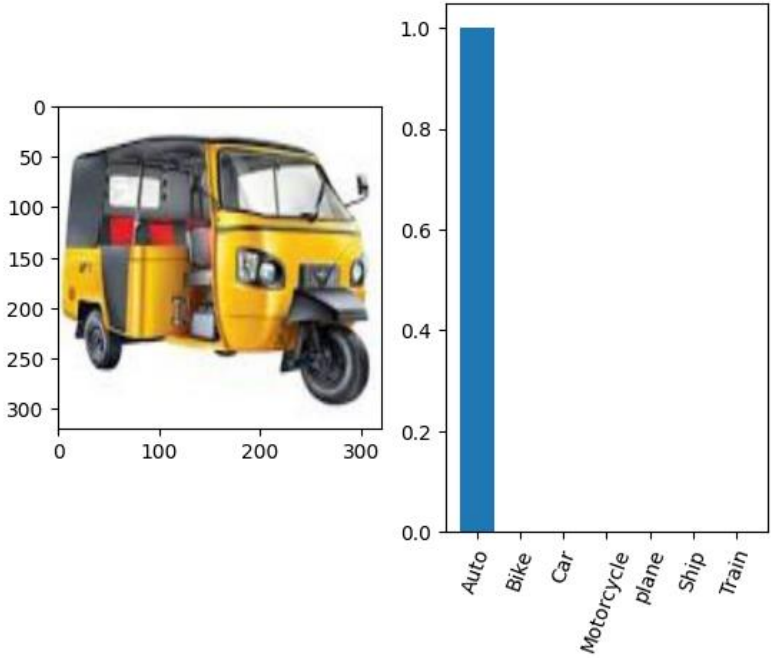
Outsource Images:

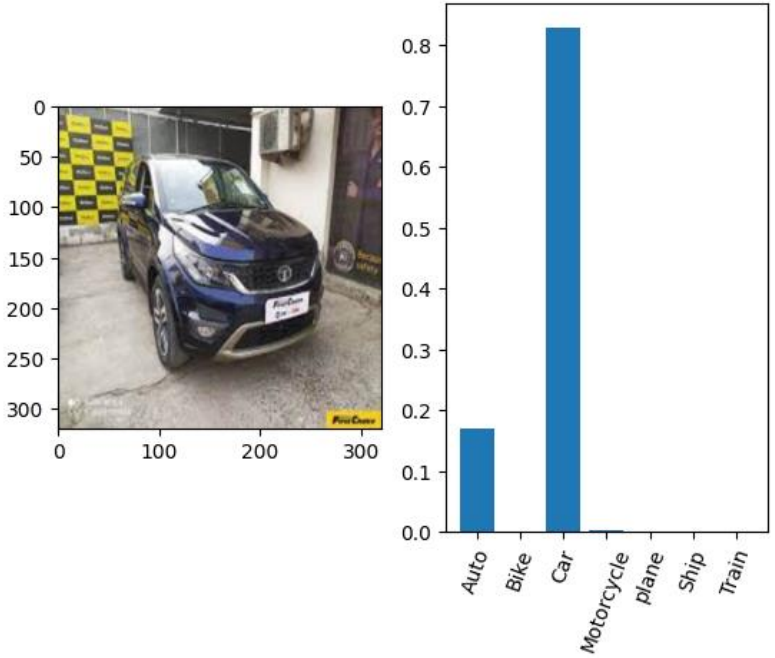
We have used image for personal items and images from internet:

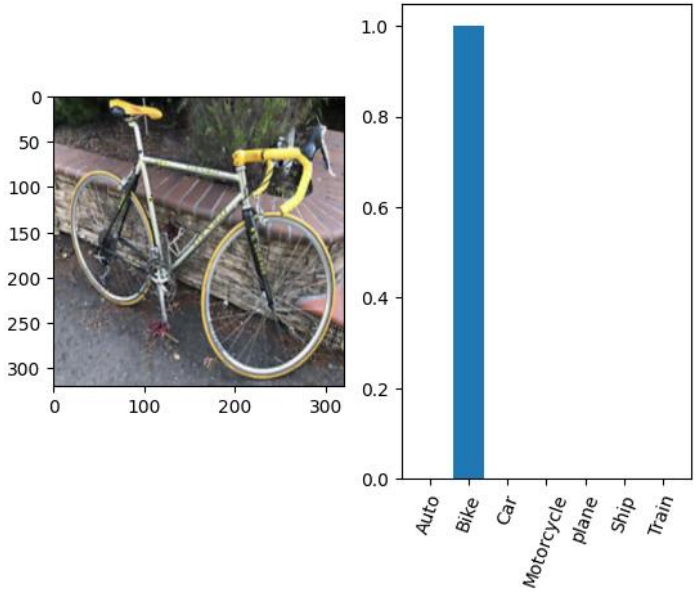
- Indian Auto
- Trains from Egypt
- Planes from Egypt
- Ship in Egypt
- Personal motorcycle

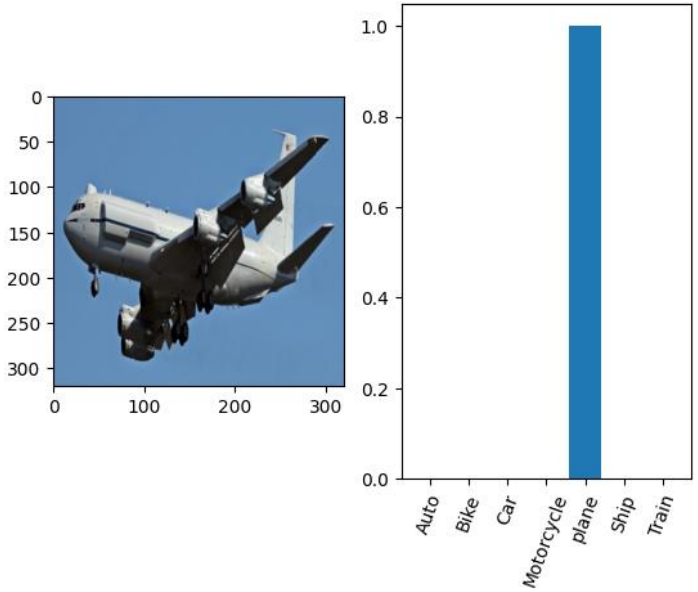
Internally from dataset:

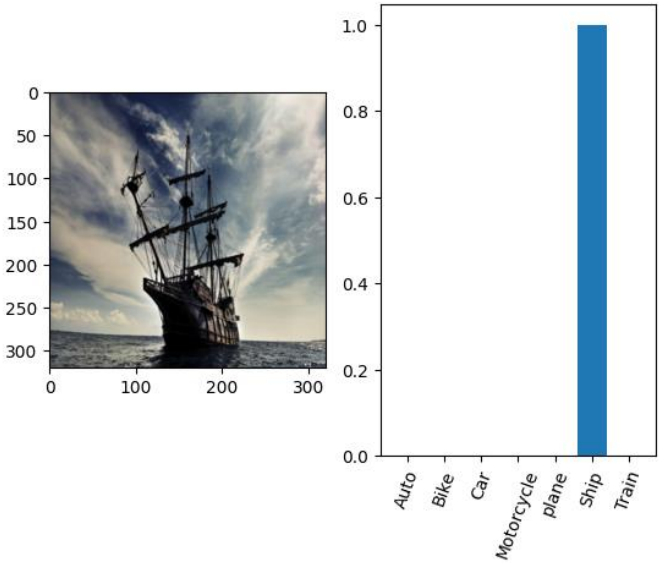


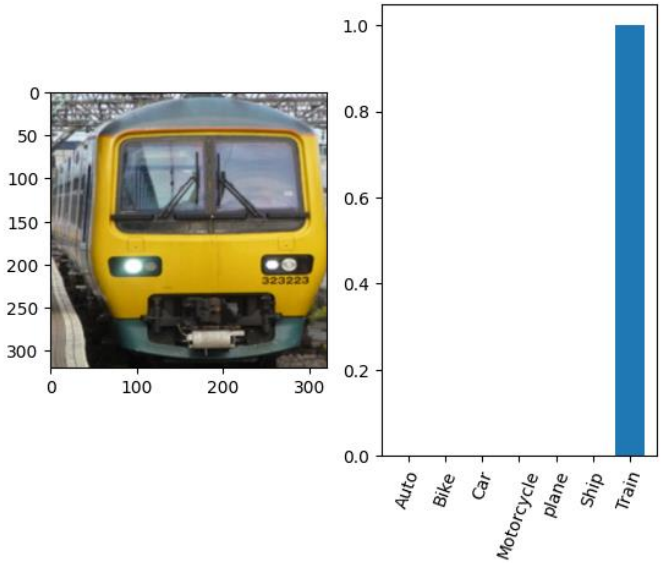




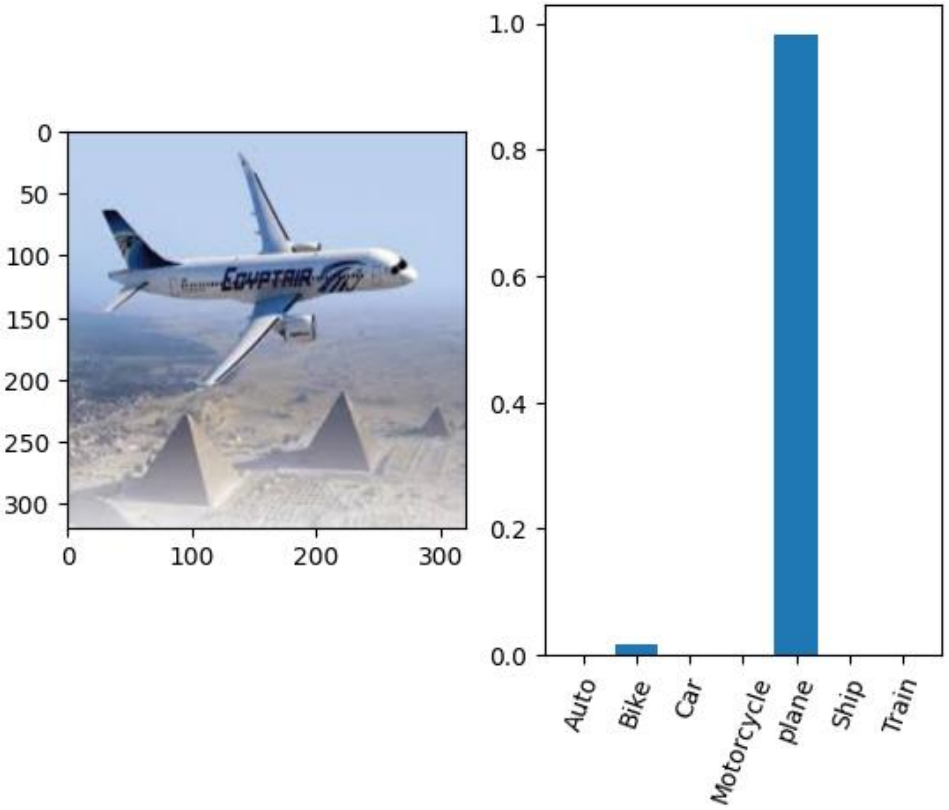

0
50
100
150
200
250
300
0
100
200
300
1.0
0.8
0.6
0.4
0.2
0.0
Auto
Bike
Car
Motorcycle
plane
Ship
Train

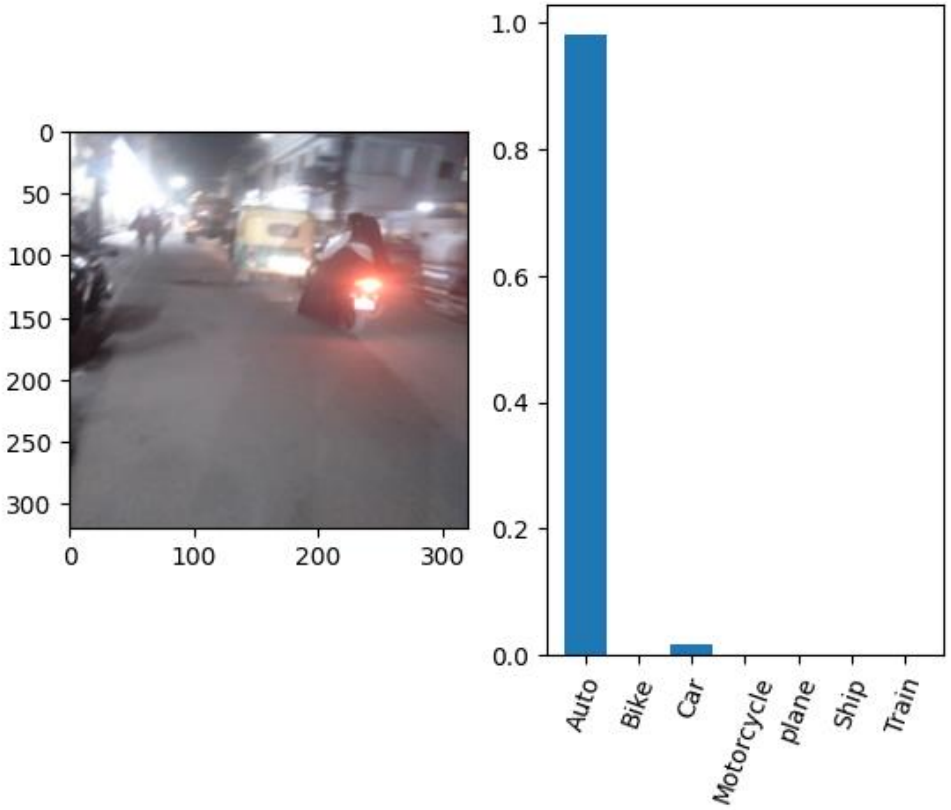

0
50
100
150
200
250
300
0
100
200
300
1.0
0.8
0.6
0.4
0.2
0.0
Auto
Bike
Car
Motorcycle
plane
Ship
Train

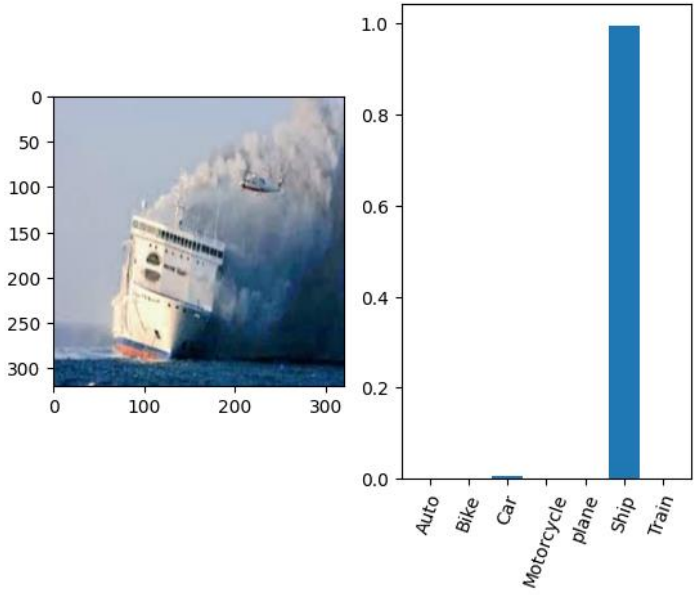

0
50
100
150
200
250
300
0
100
200
300
1.0
0.8
0.6
0.4
0.2
0.0
Auto
Bike
Car
Motorcycle
plane
Ship
Train

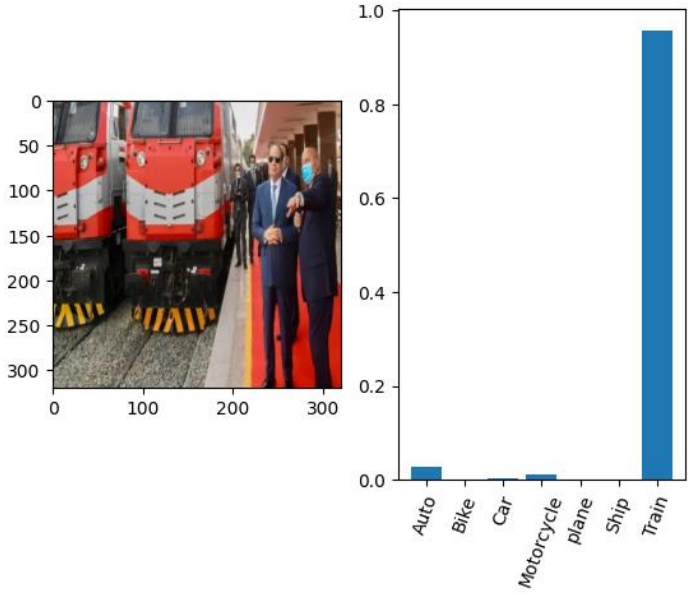

0
50
100
150
200
250
300
0
100
200
300
1.0
0.8
0.6
0.4
0.2
0.0
Auto
Bike
Car
Motorcycle
plane
Ship
Train

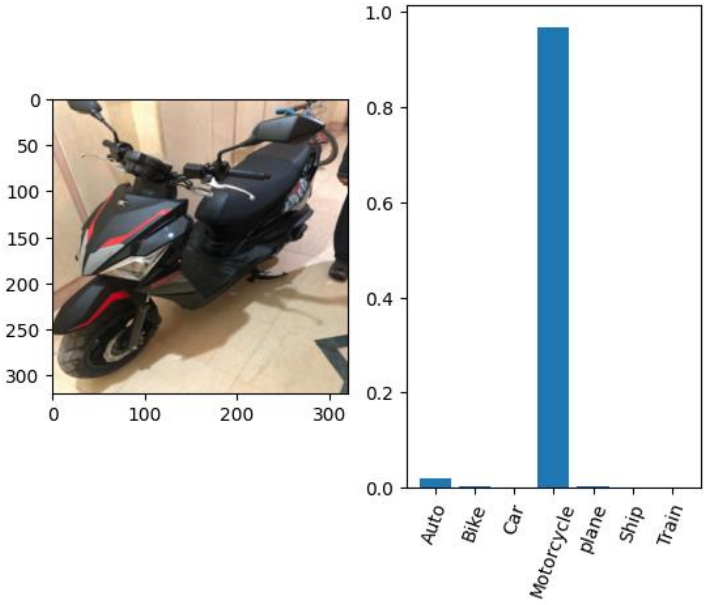
Outsource Images:

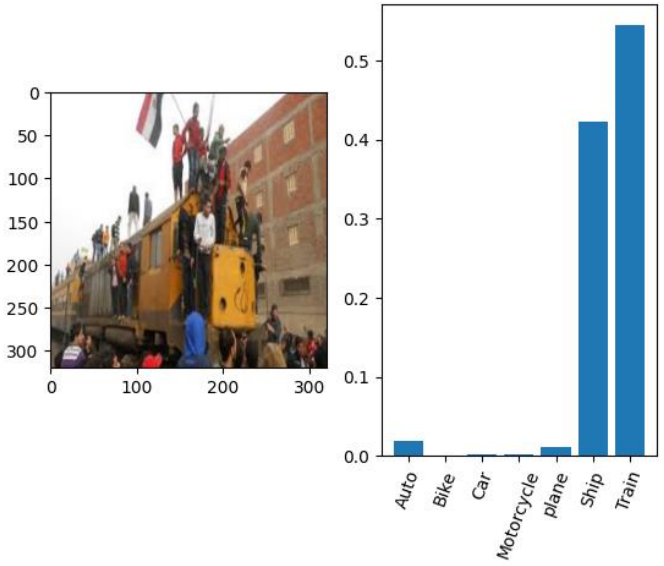





Auto
Bike
Car
Motorcycle
plane
Ship
Train


Auto
Bike
Car
Motorcycle
plane
Ship
Train


Auto
Bike
Car
Motorcycle
plane
Ship
Train


Auto
Bike
Car
Motorcycle
plane
Ship
Train

Real Case Study

Transpo Tracker by StellarView.ai

https://www.youtube.com/watch?v=JNuYSOn-VnQ

Thanks for your listening!
Any Question?